Razorpay Payment Integration Theory - Residentia Project

Complete Interview Guide - Payment Request Flow

🎯 What is Razorpay?

Razorpay is a payment gateway that enables businesses to accept online payments in India. It supports:

- Credit/Debit Cards
- Net Banking
- UPI (Google Pay, PhonePe, Paytm)
- Wallets

Why Razorpay?

- Easy integration with React and Spring Boot
- Secure payment processing
- PCI-DSS compliant
- No need to handle sensitive card data directly
- Test mode for development

🏗️ Payment Architecture Overview

```
┌─────────────────────────────────────────────────────────────┐
│                    CLIENT BROWSER                           │
│                   (React Component)                         │
└─────────────────────────────────────────────────────────────┘
                  │
                  │
                  │ 1. Payment init request
                  │
                  ▼
┌─────────────────────────────────────────────────────────────┐
│                SPRING BOOT BACKEND                          │
│          POST /api/payment/create-order                     │
│                                                             │
│  2. PaymentController creates Razorpay order                │
└─────────────────────────────────────────────────────────────┘
                  │
                  │
                  │ 3. Create order API call
                  │
                  ▼
┌─────────────────────────────────────────────────────────────┐
│               RAZORPAY SERVER                               │
│          https://api.razorpay.com                           │
│                                                             │
```

```
│  4. Returns order_id and amount                                  │
└──────────────────┬──────────────────────────────────────────────┘
                   │
                   │ 5. order_id sent to frontend
                   │
                   ▼
┌──────────────────────────────────────────────────────────────────┐
│                CLIENT BROWSER                                    │
│          Opens Razorpay Checkout Modal                           │
│                                                                  │
│  6. User selects payment method and pays                         │
└──────────────────┬──────────────────────────────────────────────┘
                   │
                   │ 7. Payment processed
                   │
                   ▼
┌──────────────────────────────────────────────────────────────────┐
│               RAZORPAY SERVER                                    │
│          Processes payment with bank                             │
│                                                                  │
│  8. Returns payment_id and signature                             │
└──────────────────┬──────────────────────────────────────────────┘
                   │
                   │ 9. Payment response
                   │
                   ▼
┌──────────────────────────────────────────────────────────────────┐
│                CLIENT BROWSER                                    │
│          Razorpay checkout callback                              │
│                                                                  │
│  10. Send payment details to backend for verification            │
└──────────────────┬──────────────────────────────────────────────┘
                   │
                   │ 11. Verify payment signature
                   │
                   ▼
┌──────────────────────────────────────────────────────────────────┐
│                SPRING BOOT BACKEND                               │
│          POST /api/payment/verify-payment                        │
│                                                                  │
│  12. Verify signature using Razorpay secret key                  │
│  13. Update booking status to PAID/CONFIRMED                     │
│  14. Send confirmation email to user                             │
└──────────────────────────────────────────────────────────────────┘
```

🔄 Complete Payment Flow - Step by Step

STEP 1: User Initiates Payment (Frontend)

Scenario: User has created a booking and needs to pay

```
┌─────────────────────────────────────────────────────────────┐
│  User Action: Client Dashboard → My Bookings → Pay Now      │
│  Component: ClientBookingPage.jsx                           │
│  URL: http://localhost:5173/payment/123                     │
│                                                             │
│  User clicks "Pay Now" button for booking ID 123            │
└─────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│  React Navigation                                           │
│  navigate(`/payment/${bookingId}`);                         │
│                                                             │
│  → Opens Payment.jsx component                              │
└─────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│  Payment.jsx Component Loads                                │
│  File: src/pages/Payment.jsx                                │
│                                                             │
│  useEffect(() => {                                          │
│      loadBookingAndInitiatePayment();                       │
│  }, [bookingId]);                                           │
└─────────────────────────────────────────────────────────────┘
```

STEP 2: Create Razorpay Order (Backend)

```
┌─────────────────────────────────────────────────────────────┐
│  Frontend API Call                                          │
│  File: src/api/bookingApi.js                                │
│                                                             │
│  const response = await axios.post(                         │
│      `${BASE_URL}/api/payment/create-order/${bookingId}`,   │
│      {},                                                    │
│      {                                                      │
│          headers: {                                         │
│              Authorization: `Bearer ${token}`               │
│          }                                                  │
│      }                                                      │
│  );                                                         │
└─────────────────────────────────────────────────────────────┘
                    │
                    │
                    │ HTTP POST Request
                    │ POST /api/payment/create-order/123
                    │ Headers: Authorization: Bearer <JWT>
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
```

```
│  Spring Security Filter Chain                                  │
│  - Validates JWT token                                         │
│  - Authenticates user                                          │
│  - Allows request to proceed                                   │
└────────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌────────────────────────────────────────────────────────────────┐
│  PaymentController.createOrder()                               │
│  File: com.residentia.controller.PaymentController.java    │
│                                                                │
│  @PostMapping("/create-order/{bookingId}")                     │
│  public ResponseEntity<?> createOrder(                         │
│      @PathVariable Long bookingId                              │
│  ) {                                                           │
│      // 1. Find booking from database                          │
│      Booking booking = bookingRepository.findById(bookingId)│
│          .orElseThrow(() -> new RuntimeException(              │
│              "Booking not found"                               │
│          ));                                                   │
│                                                                │
│      // 2. Create Razorpay client                              │
│      RazorpayClient razorpayClient = new RazorpayClient(    │
│          razorpayKeyId,      // rzp_test_S8v64jsUOHFb42     │
│          razorpayKeySecret   // 0kQhTEbbiZpkAXgnOMIST0jj    │
│      );                                                        │
│                                                                │
│      // 3. Prepare order request                               │
│      JSONObject orderRequest = new JSONObject();               │
│      orderRequest.put("amount",                                │
│          (int)(booking.getAmount() * 100) // ₹8000 = 800000 paise│
│      );                                                        │
│      orderRequest.put("currency", "INR");                      │
│      orderRequest.put("receipt", "booking_123");               │
│                                                                │
│      // 4. Create order on Razorpay                            │
│      Order order = razorpayClient.orders.create(orderRequest);│
└────────────────────────────────────────────────────────────────┘
                    │
                    │
                    │ API call to Razorpay
                    │ POST https://api.razorpay.com/v1/orders
                    │
                    ▼
┌────────────────────────────────────────────────────────────────┐
│  Razorpay Server Response                                      │
│  {                                                             │
│      "id": "order_NqPJXP9xqELPvM",                          │
│      "amount": 800000,                                         │
│      "currency": "INR",                                        │
│      "receipt": "booking_123",                                 │
│      "status": "created"                                       │
│  }                                                             │
└────────────────────────────────────────────────────────────────┘
                    │
                    │
```

```
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  PaymentController (continued)                               │
│                                                              │
│      // 5. Save order_id to database                         │
│      booking.setRazorpayOrderId("order_NqPJXP9xqELPvM");     │
│      booking.setPaymentStatus("PENDING");                    │
│      bookingRepository.save(booking);                        │
│                                                              │
│      // 6. Return response to frontend                       │
│      Map<String, Object> response = new HashMap<>();         │
│      response.put("orderId", order.get("id"));               │
│      response.put("amount", booking.getAmount());            │
│      response.put("currency", "INR");                        │
│      response.put("keyId", razorpayKeyId);                   │
│      response.put("bookingId", bookingId);                   │
│      response.put("tenantName", booking.getTenantName());    │
│      response.put("tenantEmail", booking.getTenantEmail());  │
│                                                              │
│      return ResponseEntity.ok(response);                     │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    │ HTTP 200 OK Response
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  Frontend Receives Order Details                             │
│  File: Payment.jsx                                           │
│                                                              │
│  const response = await createOrder(bookingId);              │
│  // response contains: orderId, amount, keyId, etc.          │
└──────────────────────────────────────────────────────────────┘
```

STEP 3: Open Razorpay Checkout Modal (Frontend)

```
┌──────────────────────────────────────────────────────────────┐
│  Payment.jsx - Open Razorpay Checkout                        │
│                                                              │
│  const options = {                                           │
│      key: response.keyId,  // rzp_test_S8v64jsUOHFb42        │
│      amount: response.amount * 100,  // Amount in paise      │
│      currency: "INR",                                        │
│      name: "Residentia PG",                                  │
│      description: `Booking Payment - ${response.propertyName}`,│
│      order_id: response.orderId,  // order_NqPJXP9xqELPvM    │
│      prefill: {                                              │
│          name: response.tenantName,                          │
│          email: response.tenantEmail,                       │
│          contact: response.tenantPhone                       │
```

```
│      },                                                    │
│      theme: {                                              │
│          color: "#3399cc"                                  │
│      },                                                    │
│      handler: function (paymentResponse) {                 │
│          // Called when payment succeeds                   │
│          handlePaymentSuccess(paymentResponse);            │
│      },                                                    │
│      modal: {                                              │
│          ondismiss: function() {                           │
│              // User closed modal without paying            │
│              alert("Payment cancelled");                   │
│          }                                                 │
│      }                                                     │
│  };                                                        │
│                                                            │
│  // Open Razorpay modal                                    │
│  const rzp = new window.Razorpay(options);                 │
│  rzp.open();                                               │
└────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌────────────────────────────────────────────────────────────┐
│  Razorpay Checkout Modal Opens                             │
│                                                            │
│  ┌──────────────────────────────────────────────────────┐  │
│  │         Razorpay Secure Checkout                     │  │
│  │                                                      │  │
│  │  Pay ₹8,000 to Residentia PG                         │  │
│  │                                                      │  │
│  │  [UPI]  [Card]  [NetBanking]  [Wallet]               │  │
│  │                                                      │  │
│  │  Enter Card Details:                                 │  │
│  │  Card Number: [_________________]                    │  │
│  │  Expiry: [__/__]  CVV: [___]                         │  │
│  │                                                      │  │
│  │           [Pay ₹8,000]                               │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  User selects payment method (e.g., UPI) and completes     │
└────────────────────────────────────────────────────────────┘
```

STEP 4: Payment Processing (Razorpay Server)

```
┌────────────────────────────────────────────────────────────┐
│  User Completes Payment                                    │
│  - Enters UPI PIN / Card details / Net Banking credentials │
│  - Clicks "Pay"                                            │
└────────────────────────────────────────────────────────────┘
                    │
                    │
```

```
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  Razorpay Communicates with Bank/UPI                         │
│  - Verifies credentials                                      │
│  - Debits amount from user's account                         │
│  - Credits to merchant (Residentia) account                  │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  Razorpay Generates Payment Response                         │
│  {                                                           │
│      "razorpay_payment_id": "pay_NqPK7qF1xZLv8K",            │
│      "razorpay_order_id": "order_NqPJXP9xqELPvM",            │
│      "razorpay_signature": "a1b2c3d4e5f6..."                 │
│  }                                                           │
│                                                              │
│  Signature = HMAC_SHA256(                                    │
│      order_id + "|" + payment_id,                            │
│      secret_key                                              │
│  )                                                           │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    │ Payment success callback
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  Razorpay Checkout handler() function called                 │
│  File: Payment.jsx                                           │
│                                                              │
│  handler: function (paymentResponse) {                       │
│      // paymentResponse contains:                            │
│      // - razorpay_payment_id                                │
│      // - razorpay_order_id                                  │
│      // - razorpay_signature                                 │
│                                                              │
│      handlePaymentSuccess(paymentResponse);                  │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
```

STEP 5: Verify Payment Signature (Backend)

Why Signature Verification?

- **Security:** Ensures payment response genuinely came from Razorpay
- **Prevents fraud:** Attacker can't fake a payment success
- **Signature** = HMAC hash of `order_id|payment_id` using secret key

```
┌──────────────────────────────────────────────────────────────┐
│  Frontend Sends Payment Details to Backend                   │
```

```
│  File: Payment.jsx                                          │
│                                                             │
│  const handlePaymentSuccess = async (response) => {         │
│      const verificationData = {                             │
│          razorpay_payment_id: response.razorpay_payment_id, │
│          razorpay_order_id: response.razorpay_order_id,     │
│          razorpay_signature: response.razorpay_signature    │
│      };                                                     │
│                                                             │
│      await axios.post(                                      │
│          `${BASE_URL}/api/payment/verify-payment`,          │
│          verificationData                                   │
│      );                                                     │
│  };                                                         │
└─────────────────────────────────────────────────────────────┘
                    │
                    │
                    │ HTTP POST Request
                    │ POST /api/payment/verify-payment
                    │ Body: {payment_id, order_id, signature}
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│  PaymentController.verifyPayment()                          │
│  File: PaymentController.java                               │
│                                                             │
│  @PostMapping("/verify-payment")                            │
│  public ResponseEntity<?> verifyPayment(                    │
│      @RequestBody Map<String, String> paymentData           │
│  ) {                                                        │
│      String razorpayOrderId = paymentData.get("razorpay_order_id");│
│      String razorpayPaymentId = paymentData.get("razorpay_payment_id");│
│      String razorpaySignature = paymentData.get("razorpay_signature");│
│                                                             │
│      // 1. Verify signature using Razorpay Utils           │
│      JSONObject options = new JSONObject();                 │
│      options.put("razorpay_order_id", razorpayOrderId);     │
│      options.put("razorpay_payment_id", razorpayPaymentId); │
│      options.put("razorpay_signature", razorpaySignature);  │
│                                                             │
│      boolean isValidSignature =                             │
│          Utils.verifyPaymentSignature(options, razorpayKeySecret);│
└─────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌─────────────────────────────────────────────────────────────┐
│  Signature Verification Logic (Inside Razorpay SDK)         │
│                                                             │
│  // Razorpay SDK internally does:                           │
│  String expectedSignature = HMAC_SHA256(                    │
│      razorpayOrderId + "|" + razorpayPaymentId,             │
│      razorpayKeySecret                                      │
│  );                                                         │
│                                                             │
│  if (expectedSignature.equals(razorpaySignature)) {         │
```

```
│      return true;  // ☑ Valid payment                    │
│  } else {                                                │
│      return false; // ❌ Fraudulent/tampered response     │
│  }                                                       │
└──────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────┐
│  PaymentController (continued)                           │
│                                                          │
│      if (isValidSignature) {                             │
│          // 2. Find booking by order_id                  │
│          Booking booking = bookingRepository             │
│              .findByRazorpayOrderId(razorpayOrderId)     │
│              .orElseThrow(() -> new RuntimeException(    │
│                  "Booking not found"                     │
│              ));                                         │
│                                                          │
│          // 3. Update booking with payment details       │
│          booking.setRazorpayPaymentId(razorpayPaymentId);│
│          booking.setRazorpaySignature(razorpaySignature);│
│          booking.setPaymentStatus("PAID");               │
│          booking.setStatus("CONFIRMED");                 │
│          bookingRepository.save(booking);                │
│                                                          │
│          // 4. Send confirmation email                   │
│          emailService.sendPaymentConfirmationEmail(      │
│              booking,                                    │
│              razorpayPaymentId                           │
│          );                                              │
│                                                          │
│          // 5. Return success response                   │
│          return ResponseEntity.ok(Map.of(                │
│              "success", true,                            │
│              "message", "Payment verified successfully"  │
│          ));                                             │
│      } else {                                            │
│          // ❌ Invalid signature - fraud attempt         │
│          return ResponseEntity.status(HttpStatus.BAD_REQUEST)│
│              .body(Map.of(                               │
│                  "success", false,                       │
│                  "message", "Invalid payment signature"  │
│              ));                                         │
│      }                                                   │
│  }                                                       │
└──────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────┐
│  Database Updated                                        │
│  bookings table:                                         │
│  - razorpay_order_id: "order_NqPJXP9xqELPvM"             │
│  - razorpay_payment_id: "pay_NqPK7qF1xZLv8K"             │
│  - razorpay_signature: "a1b2c3d4e5f6..."                 │
```

```
│  - payment_status: "PAID"                                    │
│  - status: "CONFIRMED"                                       │
└──────────────────────────────────────────────────────────────┘
```

STEP 6: Email Confirmation

```
┌──────────────────────────────────────────────────────────────┐
│  EmailService.sendPaymentConfirmationEmail()                 │
│  File: com.residentia.service.EmailService.java             │
│                                                              │
│  public void sendPaymentConfirmationEmail(                   │
│      Booking booking,                                        │
│      String paymentId                                        │
│  ) {                                                         │
│      String subject = "Payment Confirmation - Residentia";   │
│      String body = String.format(                            │
│          "Dear %s,\n\n" +                                    │
│          "Your payment of ₹%.2f has been successful.\n\n" +  │
│          "Booking ID: %d\n" +                                │
│          "Payment ID: %s\n" +                                │
│          "Property: %s\n\n" +                                │
│          "Thank you for choosing Residentia!",               │
│          booking.getTenantName(),                            │
│          booking.getAmount(),                                │
│          booking.getId(),                                    │
│          paymentId,                                          │
│          booking.getProperty().getName()                     │
│      );                                                      │
│                                                              │
│      // Send email via SMTP (Gmail)                          │
│      mailSender.send(                                        │
│          booking.getTenantEmail(),                           │
│          subject,                                            │
│          body                                                │
│      );                                                      │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                   │
                   │
                   ▼
┌──────────────────────────────────────────────────────────────┐
│  User Receives Email                                         │
│  To: client@example.com                                      │
│  Subject: Payment Confirmation - Residentia                  │
│                                                              │
│  Dear John Doe,                                              │
│                                                              │
│  Your payment of ₹8,000.00 has been successful.             │
│                                                              │
│  Booking ID: 123                                             │
│  Payment ID: pay_NqPK7qF1xZLv8K                              │
```

```
│  Property: Sunshine PG                                          │
│                                                                 │
│  Thank you for choosing Residentia!                             │
└─────────────────────────────────────────────────────────────────┘
```

STEP 7: Frontend Handles Success

```
┌─────────────────────────────────────────────────────────────────┐
│  Payment.jsx - Success Handler                                  │
│                                                                 │
│  const response = await verifyPayment(verificationData);        │
│                                                                 │
│  if (response.success) {                                        │
│      // Show success message                                    │
│      toast.success("Payment successful! ☑");                    │
│                                                                 │
│      // Redirect to bookings page                               │
│      setTimeout(() => {                                         │
│          navigate('/client/dashboard/my-bookings');             │
│      }, 2000);                                                  │
│  }                                                              │
└─────────────────────────────────────────────────────────────────┘
```

🔑 Key Components & Configuration

1. Backend Dependencies (pom.xml)

```xml
<!-- Razorpay Java SDK -->
<dependency>
    <groupId>com.razorpay</groupId>
    <artifactId>razorpay-java</artifactId>
    <version>1.4.6</version>
</dependency>

<!-- JSON processing (for Razorpay API) -->
<dependency>
    <groupId>org.json</groupId>
    <artifactId>json</artifactId>
    <version>20231013</version>
</dependency>
```

---

2. Application Configuration (application.yml)

```yaml
# Razorpay credentials
razorpay:
  key:
    id: rzp_test_S8v64jsUOHFb42      # Public key (frontend)
    secret: 0kQhTEbbiZpkAXgnOMIST0jj  # Secret key (backend only)

# Email configuration
spring:
  mail:
    host: smtp.gmail.com
    port: 587
    username: your-email@gmail.com
    password: app-specific-password
    properties:
      mail:
        smtp:
          auth: true
          starttls:
            enable: true
```

---

3. Database Schema (Booking Entity)

```java
@Entity
@Table(name = "bookings")
public class Booking {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Double amount;

    // Razorpay fields
    private String razorpayOrderId;    // order_NqPJXP9xqELPvM
    private String razorpayPaymentId;  // pay_NqPK7qF1xZLv8K
    private String razorpaySignature;  // a1b2c3d4e5f6...

    private String paymentStatus;  // PENDING, PAID, FAILED
    private String status;         // PENDING, CONFIRMED, CANCELLED

    // Other fields...
}
```

---

4. Frontend - Razorpay Script (index.html)

```html
<!-- Load Razorpay Checkout.js from CDN -->
<script src="https://checkout.razorpay.com/v1/checkout.js"></script>
```

🔍 Common Interview Questions & Answers

Q1: Why create order on backend instead of frontend?

A:

- **Security:** Secret key must NEVER be exposed on frontend
- **Amount validation:** Backend ensures correct amount (prevents tampering)
- **Database sync:** Order ID saved to database before payment
- **Audit trail:** All orders logged server-side

Q2: What is the purpose of signature verification?

A:

- **Prevents fraud:** Ensures payment response genuinely from Razorpay
- **Data integrity:** Confirms data wasn't tampered during transit
- **HMAC signature** = Cryptographic hash that only Razorpay and our backend can generate (using secret key)

Q3: What happens if user closes payment modal?

A:

- `modal.ondismiss` callback is triggered
- Order remains in "created" state on Razorpay
- Booking remains "PENDING" in database
- User can retry payment later using same order ID

Q4: How to handle payment failures?

A:

```
handler: function (response) {
    // Success callback
},
modal: {
    ondismiss: function() {
        // User cancelled
        alert("Payment cancelled. Try again.");
    }
}
```

Backend can also check payment status via Razorpay API:

```
Payment payment = razorpayClient.payments.fetch(paymentId);
String status = payment.get("status"); // captured, failed, authorized
```

Q5: Why convert amount to paise (multiply by 100)?

A:

- Razorpay expects amount in **smallest currency unit** (paise for INR)
- ₹8,000 = 800,000 paise
- Avoids floating-point precision issues
- Standard practice for payment gateways

Q6: How to test payments in development?

A:

- Use **test mode** keys (`rzp_test_...`)
- Test cards provided by Razorpay:
  - Card: 4111 1111 1111 1111
  - CVV: Any 3 digits
  - Expiry: Any future date
- Test UPI: success@razorpay
- No real money is charged

Q7: What is the flow of money?

A:

```
User's Bank Account
    ↓ (debit ₹8,000)
Razorpay Escrow Account (holds temporarily)
    ↓ (settlement after T+2 days)
Merchant Bank Account (Residentia)
```

Q8: How to handle refunds?

A:

```
// Create refund via Razorpay API
Payment payment = razorpayClient.payments.fetch(paymentId);
payment.refund(new JSONObject().put("amount", 800000)); // Full refund
```

```
// Or partial refund
payment.refund(new JSONObject().put("amount", 400000)); // ₹4,000
```

📊 Payment Status Flow Diagram

```
┌──────────────┐
│   BOOKING    │ ← User creates booking
│   CREATED    │
└──────────────┘
       │
       ▼
┌──────────────┐
│  RAZORPAY    │ ← Backend creates Razorpay order
│   ORDER      │
│   CREATED    │
└──────────────┘
       │
       ▼
┌──────────────┐
│   PAYMENT    │ ← User completes payment on Razorpay modal
│   CAPTURED   │
└──────────────┘
       │
       ▼
┌──────────────┐
│  SIGNATURE   │ ← Backend verifies signature
│  VERIFIED    │
└──────────────┘
       │
       ▼
┌──────────────┐
│   BOOKING    │ ← Update booking status
│  CONFIRMED   │
└──────────────┘
       │
       ▼
┌──────────────┐
│    EMAIL     │ ← Send confirmation email
│    SENT      │
└──────────────┘
```

🎯 Summary for Interviews

Key Concepts to Remember:

1. **Server-side order creation** for security (secret key never exposed)
2. **Razorpay Checkout** handles payment UI (no need to collect card details)
3. **Signature verification** ensures payment authenticity

4. **Amount in paise** to avoid floating-point issues
5. **Test mode** for development (no real charges)
6. **Email confirmation** sent after successful payment
7. **Database updates** happen only after signature verification
8. **Three-way handshake:** Frontend → Backend → Razorpay → Backend → Frontend

What makes this implementation production-ready:

- ☑ Secure (secret key on backend only)
- ☑ PCI-DSS compliant (card data handled by Razorpay)
- ☑ Signature verification (prevents fraud)
- ☑ Database consistency (atomic updates)
- ☑ Error handling (try-catch blocks)
- ☑ Logging (audit trail)
- ☑ Email notifications

End of Payment Theory Document